



Fundação Presidente Antônio Carlos

Engenharia de Computação

**Leis de Amdahl e Gustafson na Prática:
Limitações e Potencial do Paralelismo em CPU e GPU**

Marcos Paulo Araujo Pereira

Vinicius Almeida Fajardo

Conselheiro Lafaiete - MG
2026

Sumário

1	Introdução	5
2	Problema de Pesquisa	5
3	Hipóteses	5
4	Objetivos	6
4.1	Objetivo Geral	6
4.2	Objetivos Específicos	6
5	Fundamentação Teórica	7
5.1	Lei de Amdahl	7
5.2	Lei de Gustafson	8
5.3	CUDA e OpenMP	9
5.4	Trabalhos Relacionados	13
6	Metodologia	14
6.1	Materiais	14
6.2	Estrutura do Benchmark	15
6.3	Procedimento Experimental	17
6.4	Análise dos Dados	18
7	Resultados Esperados	19
8	Cronograma	19
9	Referências	21

Lista de Figuras

1	Curva teórica da Lei de Amdahl $speedup\ S(N)$ em função do número de processadores N para diferentes valores da fração paralela P (50%, 75%, 90%, 95%). Demonstra a saturação do ganho quando a fração serial domina, limitando o $speedup$ máximo a $1/(1 - P)$. Gráfico gerado no Google Colab.	7
2	Curva teórica da Lei de Gustafson $speedup$	9
3	Declaração de variável armazenada na GPU com <code>__device__</code>	10
4	Função <code>__device__</code> armazenada na VRAM e invocada internamente pela GPU.	10
5	<i>Kernel</i> CUDA de Monte Carlo simples: cada <i>thread</i> executa seu próprio laço e contribui via <code>atomicAdd</code>	11
6	Laço <code>for</code> sequencial sem OpenMP.	12
7	Mesmo laço paralelizado com a diretiva <code>#pragma omp parallel for</code>	12
8	Solução para a <i>race condition</i> usando a cláusula <code>reduction</code>	12
9	Configurações de hardware A, B e C utilizadas no benchmark CPU (modelo, núcleos físicos, <i>threads</i> e <i>clock</i> base), memória RAM (capacidade e padrão) e GPU (modelo, CUDA cores, VRAM e geração PCIe). Diagrama feito no draw.io a partir das especificações oficiais das fabricantes (AMD, Intel e NVIDIA).	15
10	Definição da estrutura <code>Event</code> alinhada em 96 <i>bytes</i>	16
11	Ilustração do encaixe da estrutura na linha de transferência (feito no draw.io).	17

Lista de Tabelas

1	Cronograma de execução do projeto.	20
---	--	----

Lista de Equações

1	Speedup de Amdahl	7
2	Eficiência de Amdahl	8
3	Speedup de Gustafson	8
4	Eficiência de Gustafson	9

1 Introdução

Computação paralela é uma técnica computacional de dividir problemas complexos em segmentos menores com finalidade de aproveitar os vários elementos computacionais presentes no sistema, atualmente possui alta demanda computacional em data centers para diversas finalidades como treinamento de modelos de inteligência artificial, simulações científicas, processamentos de grandes volumes de dados e outros sistemas, isso tudo exige paralelismo a fim de se tornar mais eficiente na entrega de resultados em menor tempo e obter eficiência energética mantendo os elementos computacionais mais ocupados possível a cada clock [2].

Este projeto busca demonstrar os cenários nos quais a computação paralela faz sentido através do uso de um Benchmark que será desenvolvido, além de abranger diferentes módulos como serial, paralelo em CPU (OpenMP) e paralelo em GPU (CUDA), serão abordados diferentes algoritmos de busca como linear, binária e hash e algoritmos matemáticos (Monte Carlo e Mandelbrot), demonstrando pelos resultados obtidos como é importante a integração entre hardware e software, toda a simulação é baseada nos limites teóricos previstos pelas leis de Amdahl e Gustafson.

2 Problema de Pesquisa

Em que medida as leis de Amdahl e Gustafson são capazes de explicar, na prática, o ganho de desempenho (*speedup*) obtido por algoritmos distintos com mesmo objetivo, executados em CPU serial, CPU paralela (OpenMP) e GPU (CUDA), e como o volume de dados influencia esse comportamento?

3 Hipóteses

- **H1:** Em busca linear, para volumes de dados pequenos em relação ao poder de processamento, o custo de criação e sincronização de threads para dividir tarefas supera o processamento sequencial de um único núcleo, pois o tempo de organização, a divisão de tarefas e a fusão de resultados é maior do que o tempo de processamento direto. A partir de determinado volume de dados, o tempo gasto em organização e fusão dos resultados torna-se aceitável, pois passa a ser menor do que o tempo que um único núcleo levaria para processar aquele volume.
- **H2:** A busca por hash, quanto maior é o volume de dados, maior é a perda de eficiência devido às colisões de hash. A partir das colisões, são utilizadas busca serial ou binária, podendo tornar essa parte a fração serial conforme a lei de Amdahl. Caso essa colisão consiga ser tratada como busca binária, otimizamos tanto o algoritmo que a fração serial se torna eficiente. Em resumo, uma função hash deve possuir o menor nível de colisão possível para se tornar muito eficiente para volumes de dados grandes.

- **H3:** Diferente dos algoritmos de busca, algoritmos de matemática densa como Monte Carlo e Mandelbrot são tão complexos que o tempo de transferência via PCIe e o tempo gasto na divisão e organização das informações tornam-se desprezíveis diante do volume de cálculo, tornando-os muito viáveis para processamento em GPUs. Por outro lado, representam um gargalo significativo quando processados em CPUs, devido ao baixo número de núcleos. Por serem altamente paralelizáveis, esses algoritmos compensam a latência de transferência via PCIe e o tempo gasto na divisão e organização das informações [7].

4 Objetivos

4.1 Objetivo Geral

Demonstrar na prática as leis de Amdahl e Gustafson através de um benchmark próprio, executado em diferentes configurações de hardware, variando tanto a capacidade da CPU quanto da GPU, comparando algoritmos com finalidades distintas — busca e matemática densa — executados em diferentes módulos: CPU serial, CPU paralela (OpenMP) e GPU (CUDA), a fim de demonstrar onde a técnica de processamento paralelo pode ser eficiente e como os barramentos de memória são um gargalo.

4.2 Objetivos Específicos

- Implementar a estrutura `Event` alinhada (96 bytes) e o gerador sintético de dados (Box-Muller, Poisson, checksum XOR).
- Implementar busca linear, binária e hash nos três modos de execução.
- Implementar Monte Carlo (estimativa de π) e Mandelbrot nos três modos.
- Medir tempos por meio de medianas de múltiplas repetições e calcular *speedup* e eficiência.
- Discutir os resultados frente às leis de Amdahl e Gustafson, identificando regimes *compute bound* e *memory bound*.
- Compilar o benchmark com parâmetros específicos para cada arquitetura-alvo, garantindo que o código gerado utilize as instruções nativas de cada hardware, como as arquiteturas Turing (GTX 16xx) e Ampere (RTX 30xx).

5 Fundamentação Teórica

5.1 Lei de Amdahl

Lei de Amdahl: A Perspectiva de Carga Fixa. Criada por Gene Amdahl em 1967 [13], a Lei de Amdahl assume que o tamanho do problema é fixo, com um simples objetivo de responder a questão de quanto tempo podemos economizar ao executar uma tarefa de tamanho fixo utilizando mais processadores? Qualquer programa possui uma fração de código que pode ser executada de forma estritamente paralela (P) e uma porção que é necessariamente sequencial ou serial ($1 - P$), como rotinas de inicialização, operações de entrada/saída (E/S) e sincronização de threads.

Fórmula do Speedup de Amdahl. O ganho de velocidade teórico (*speedup*) obtido ao distribuir o código por N processadores é expresso por:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}} \quad (1)$$

onde P = quantidade do código em paralelo e N = quantidade de processadores.

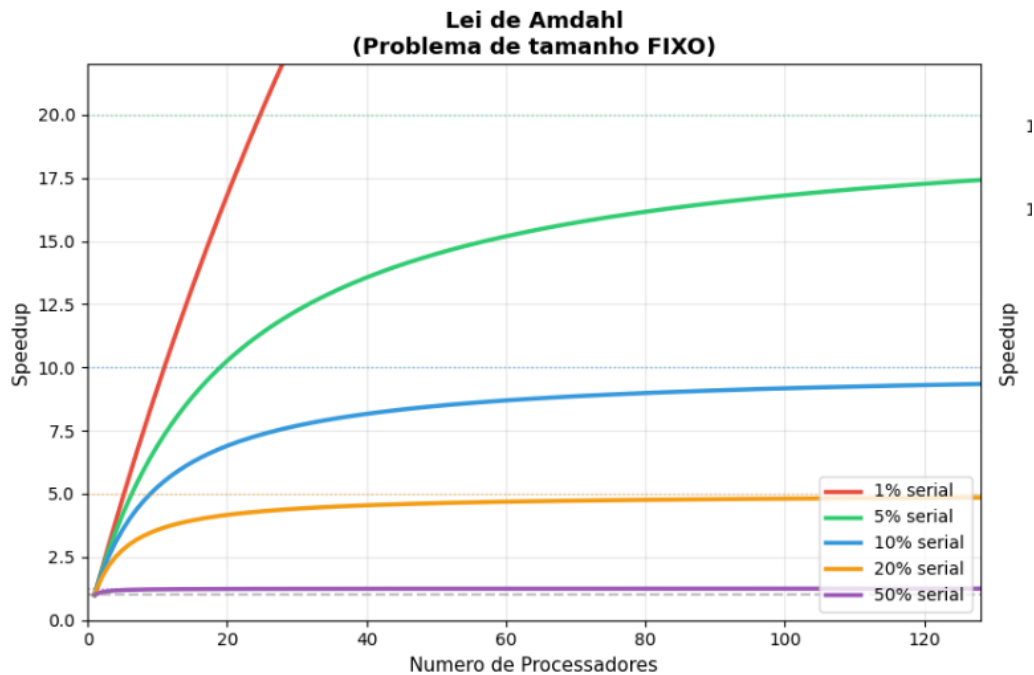


Figura 1: Curva teórica da Lei de Amdahl $speedup\ S(N)$ em função do número de processadores N para diferentes valores da fração paralela P (50%, 75%, 90%, 95%). Demonstra a saturação do ganho quando a fração serial domina, limitando o *speedup* máximo a $1/(1 - P)$. Gráfico gerado no Google Colab.

Fórmula de Eficiência de Amdahl. A eficiência (E) mede o nível de aproveitamento dos

processadores disponíveis, sendo calculada a partir da divisão do *speedup* obtido pelo número de processadores N :

$$E(N) = \frac{S(N)}{N} = \frac{1}{N(1 - P) + P} \quad (2)$$

À medida que o número de recursos computacionais cresce indefinidamente (N para infinito), o termo P/N aproxima-se de zero, significando que o *speedup* máximo fica estritamente limitado pelo inverso da fração serial $1/(1 - P)$, enquanto a eficiência decai para zero (E para 0), se tratando assim de uma visão pessimista, pois mesmo que adicionemos mais núcleos a um problema fixo, teremos retornos decrescentes severos devido ao gargalo da parte serial [11].

5.2 Lei de Gustafson

Lei de Gustafson: A Perspectiva de Carga Escalável. Proposta por John Gustafson e Edwin Barsis em 1988 [14], a Lei de Gustafson flexibiliza a premissa de Amdahl, argumentando que, na prática em um ambiente computacional real, o tamanho do problema não permanece estático como é citado na Lei de Amdahl, e sim é crescente, e para melhorar a eficácia e velocidade do programa, é expandido o tamanho do problema (aumentando o volume de dados, o nível de detalhe de uma simulação ou o tamanho de um modelo de IA) para ser resolvido no mesmo intervalo de tempo original.

Fórmula do Speedup de Gustafson. O ganho de velocidade escalado tenta assumir um comportamento linear em relação ao número de recursos:

$$S(N) = (1 - P) + P \times N \quad (3)$$

onde P = tempo gasto na parte paralela e N = número de processadores utilizado.

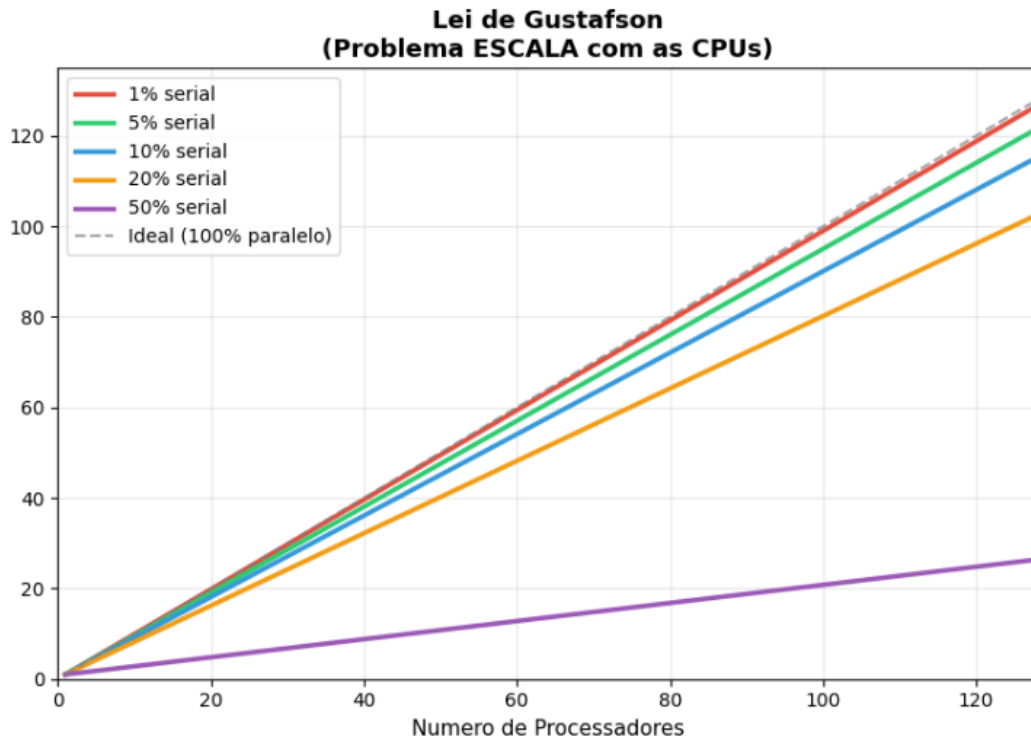


Figura 2: Curva teórica da Lei de Gustafson speedup escalado $S(N)$ em função do número de processadores N para diferentes valores da fração paralela P (50%, 75%, 90%, 95%). Diferente da Lei de Amdahl, o crescimento é aproximadamente linear, refletindo a hipótese de problema escalável. Gráfico gerado no Google Colab.

Fórmula de Eficiência de Gustafson. Dividindo o *speedup* escalado pelo número de processadores N , resulta na eficiência correspondente:

$$E(N) = \frac{S(N)}{N} = \frac{1 - P}{N} + P \quad (4)$$

Sendo assim, ao contrário do modelo de Amdahl, em que quando N tende ao infinito a eficiência tende a ser zero, a eficiência na lei de Gustafson não decai a zero, pois o termo $(1 - P)/N$ desaparece, fazendo com que a eficiência diminua para a própria fração paralela do sistema (E para P). Por exemplo, se 99% do tempo de execução de um problema expandido for em paralelo, a eficiência se manterá estável próxima a 99%, o que viabiliza altos ganhos de velocidade mais “lineares” em computadores.

5.3 CUDA e OpenMP

GPUs (Unidades de Processamento Gráfico) foram desenvolvidas como elemento de processamento dedicado para gráficos 3D. Ao longo do tempo, as GPUs começaram a possuir objetivos além do propósito pelo qual foram desenvolvidas, tornando-se mais programáveis.

Em 2006, a NVIDIA lançou a tecnologia CUDA (*Compute Unified Device Architecture*), abrindo as fronteiras de acesso aos recursos da GPU que antes eram limitados a apenas APIs gráficas. Isso possibilitou a definição de diferentes finalidades para a GPU, tirando-a de apenas gráficos 3D e trazendo-a para otimizar cargas de trabalho que são pesadas para a CPU, lembrando que “Os aplicativos CUDA executam alguma parte de seu código na GPU, mas os aplicativos sempre iniciam a execução na CPU” [4].

Uma CPU foi projetada para se destacar em fluxo de tarefas e dados sequenciais, focada em lógica e decisões. Trabalha de forma que orquestra todo o sistema, enquanto as GPUs foram projetadas para possuir muitas unidades de processamento simples e cache dedicado para agendamento de execução. Um único *warp* que se divide por causa de decisão pode afetar o desempenho da execução; como são unidades simples em grandes quantidades, elas foram otimizadas para possuírem um nível de lógica ou decisões muito mais baixo comparado com a CPU. A GPU se destaca em cálculos e transformações de dados, e a CPU se destaca na tomada das decisões que permite todo o sistema funcionar.

As instruções CUDA são utilizadas como extensão de linguagens de programação a fim de criar *kernel* que será executado na GPU. O *kernel* (Núcleo) é a definição de um trecho de código desenvolvido em linguagens como C/C++ e Python. Como dito anteriormente, ele sempre inicializa na CPU e a carga de trabalho é passada para a GPU. Um código `main.c` é executado na CPU e, ao chamar uma função *kernel*, ela é passada para a GPU. A função *kernel* não é executada apenas uma vez ela cria milhares de *threads* que executam o mesmo código, porém operando em diferentes partes de dados [4].

Instruções de Localidade

`__global__` Define que o comando a seguir é um *Kernel* que será chamado pela CPU, mas executado pela GPU.

`__device__` Define que o comando é executado apenas na GPU, como uma declaração de variável conforme o exemplo a seguir:

```
__device__ float variavel_array[8]
```

Figura 3: Declaração de variável armazenada na GPU com `__device__`.

isso define que essa variável é armazenada na GPU em sua VRAM, assim como funções:

```
__device__ float calcular_distancia(float x, float y) { return sqrt(x*x + y*y); }
```

Figura 4: Função `__device__` armazenada na VRAM e invocada internamente pela GPU.

essa função fica armazenada na VRAM e é invocada internamente na GPU sem dependência da CPU.

`__host__` Define que o comando é executado apenas na CPU. Normalmente não é necessário utilizar; o próprio código fora de `__global__` já é executado na CPU.

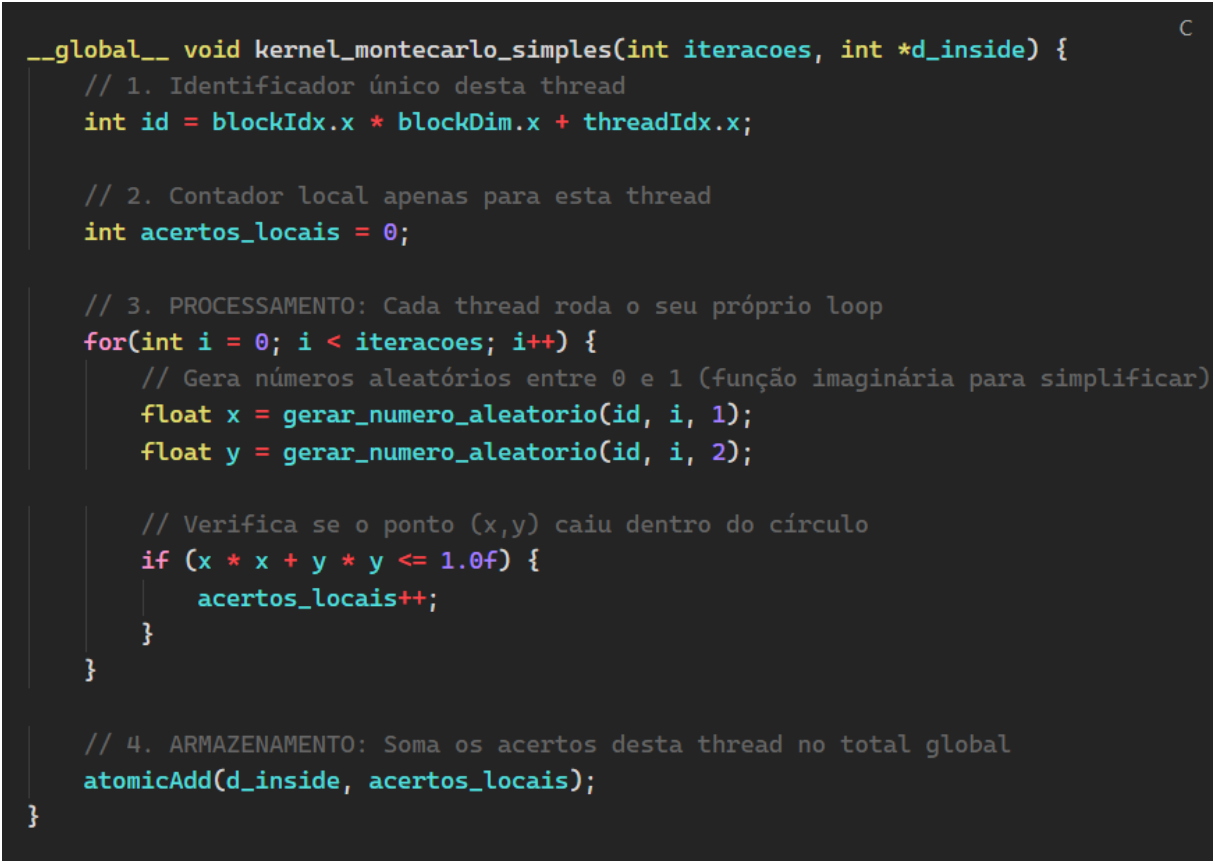
Instruções de Memória

`__shared__` Define que a memória utilizada será o cache compartilhado entre o bloco de *threads*.

`__constant__` Definição de um tipo de dado constante de fácil acesso para *threads*.

Sincronização

`__syncthreads()` — Comando para sincronizar todas as *threads* do bloco. É um comando de pausa para *threads* adiantadas que garante a sincronização de dados calculados, e é uma forma de segurança, pois *threads* que ainda não terminaram a execução podem conter dados necessários para outras *threads* continuarem.



```
__global__ void kernel_montecarlo_simples(int iteracoes, int *d_inside) {  
    // 1. Identificador único desta thread  
    int id = blockIdx.x * blockDim.x + threadIdx.x;  
  
    // 2. Contador local apenas para esta thread  
    int acertos_locais = 0;  
  
    // 3. PROCESSAMENTO: Cada thread roda o seu próprio loop  
    for(int i = 0; i < iteracoes; i++) {  
        // Gera números aleatórios entre 0 e 1 (função imaginária para simplificar)  
        float x = gerar_numero_aleatorio(id, i, 1);  
        float y = gerar_numero_aleatorio(id, i, 2);  
  
        // Verifica se o ponto (x,y) caiu dentro do círculo  
        if (x * x + y * y <= 1.0f) {  
            acertos_locais++;  
        }  
    }  
  
    // 4. ARMAZENAMENTO: Soma os acertos desta thread no total global  
    atomicAdd(d_inside, acertos_locais);  
}
```

Figura 5: *Kernel* CUDA de Monte Carlo simples: cada *thread* executa seu próprio laço e contribui via `atomicAdd`.

OpenMP (*Open Multi-Processing*) é uma API muito utilizada em programação que aproveita recursos como processadores *multi-core*. Essa API utiliza Diretivas de Compilação (*Pragmas*) que comunicam ao compilador como o código deve ser executado em paralelo [10].

Exemplo sem OpenMP:

```

for (int i = 0; i < 1000000; i++) {
    contar(caixas[i]);
}

```

Figura 6: Laço `for` sequencial sem OpenMP.

Com OpenMP:

```

#pragma omp parallel for
for (int i = 0; i < 1000000; i++) {
    contar(caixas[i]); // cada núcleo pega uma caixa diferente
}

```

Figura 7: Mesmo laço paralelizado com a diretiva `#pragma omp parallel for`.

No exemplo acima com OpenMP, o compilador lê `#pragma omp parallel for` e entende que esse `for` será dividido entre vários núcleos, automaticamente, sem que o programador necessite fazer algo a mais.

O problema do código acima é o acesso à memória compartilhada. Imagine que cada núcleo seja uma pessoa que pega a `caixa[i]`, abre ela e verifica se ela é vermelha se sim, vai à mesa e atualiza o contador. Essas pessoas possuem acesso compartilhado e podem atualizar a informação no mesmo momento, gerando um falso incremento. Isso é chamado de *race condition*.

Solução:

```

#pragma omp parallel for reduction(+:total)
for (int i = 0; i < n; i++) {
    if (e_vermelho(caixa[i]))
        total++; // cada um anota no SEU rascunho
}

```

Figura 8: Solução para a *race condition* usando a cláusula `reduction`.

Acrescentando `reduction(+:total)`, cada *thread* passa a ter sua própria memória. Automaticamente, no final do `for`, todos esses contadores são somados. Essa técnica é chamada de *reduction*: divisão do problema e junção dos resultados no final.

Por padrão é definida uma divisão estática que divide o número de iterações `n` pelo número de *threads*. Por isso é importante levar em conta que cada caixa carregada pelo contador pode ter pesos diferentes, dificultando a contagem se alguns contadores terminarem suas pilhas, vão aguardar os que ainda estão trabalhando.

Com base nisso, acrescentamos `schedule(dynamic)`: essa divisão será dinâmica, conforme o contador termina sua pilha ele pega mais, até que todas as iterações tenham finalizado.

O *chunk* padrão do *dynamic* é 1 iteração por vez — funciona, mas gera *overhead* de coordenação a cada passo. Por isso é comum usar `schedule(dynamic, 4)`, onde cada contador pega 4 caixas por vez, reduzindo as idas à “fila central” sem deixar ninguém ocioso [6].

5.4 Trabalhos Relacionados

Computação Homogênea e Heterogênea

- **Homogênea:** Usa mesmos elementos computacionais com a mesma arquitetura e capacidade, usada quando o trabalho realizado é dimensionado exclusivamente para aquela arquitetura e recurso disponível.
- **Heterogênea:** Usa elementos computacionais com arquitetura e recursos diferentes como CPUs, GPUs além de outras unidades a fim de maximizar a integração de hardware e software de forma distribuída, como exemplo acelerar trabalho usando GPUs quando fizer sentido, de forma que dimensione o trabalho conforme a complexidade [1][9].

Modelos Sun–Ni e Hill–Marty

- **Modelo Sun–Ni:** considera a velocidade e capacidade das memórias presentes, usada como base para supercomputadores e sistemas de múltiplos núcleos modernos onde o limite das memórias como *Memory wall* interfere diretamente no *speedup* final; restringe o dimensionamento do problema com base na capacidade da memória [12].
- **Modelo Hill–Marty:** é uma extensão da lei de Amdahl que considera computação heterogênea de forma que consegue calcular o *speedup* baseado em diferentes recursos disponíveis por elemento computacional. Descreve Núcleos Simétricos (vários núcleos menores e simples) e Núcleos Assimétricos (um ou poucos núcleos de alto desempenho combinados com muitos núcleos menores) [1].

O Modelo Amdahl Modernizado

O modelo clássico que será a base do benchmark deste artigo está limitado a variáveis não consideradas, como a fração serial e paralela em função do tempo, pois ao iniciar um processo ele pode sofrer taxas de mudança em suas frações seriais e paralelas [2].

Paradigmas Paralelos em HPC Moderno

- **MPI:** Paralelismo em nível de processo, como em *clusters* que possuem memórias isoladas, onde há necessidade de usar MPI para haver comunicação entre esses processos.
- **OpenMP:** Paralelismo em nível de *threads* em CPUs.
- **CUDA:** Paralelismo de dados massivos [3].

Desempenho dos Modelos de Programação CUDA, OpenACC e OpenMP na GPU TESLA V100

Análise sobre modelos de programação com testes reais em GPU potente TESLA V100, mostrando a real diferença entre desenvolver usando CUDA e OpenMP em GPUs. O estudo mostra que usar programação CUDA é uma forma mais otimizada, pois CUDA alinha a arquitetura com o software, sendo algo nativo. A avaliação e os testes mostram que OpenACC e OpenMP, como programação de alto nível, são capazes de gerenciar a GPU em casos simples, enquanto o CUDA — programação de baixo nível — é mais otimizado em casos simples e complexos [5].

6 Metodologia

6.1 Materiais

A simulação será conduzida em configurações de hardware diferentes. Até o momento temos acesso a 3 configurações:

Configuração A (Turing) *notebook*. CPU AMD Ryzen 7 4800H (8 núcleos físicos, 16 *threads*, 2,9 GHz base), 8 GB DDR4-3200, GPU NVIDIA GeForce GTX 1650 (896 CUDA cores, 4 GB GDDR6, PCIe 3.0 $\times$ 16). Sistema operacional Windows 11.

Configuração B (Ampere) *desktop*. CPU Intel i3 10105F (4 núcleos físicos, 8 *threads*, 3,7 GHz base), 8 GB DDR4-3200, GPU NVIDIA GeForce RTX 3080 (8704 CUDA cores, 10 GB GDDR6X, PCIe 4.0 $\times$ 16). Sistema operacional Windows 11.

Configuração C (Ampere) *desktop*. CPU Intel i5 12400 (6 núcleos físicos, 12 *threads*, 2,5 GHz base), 32 GB DDR4-3200, GPU NVIDIA GeForce RTX 3060 (3584 CUDA cores, 12 GB GDDR6, PCIe 4.0 $\times$ 16). Sistema operacional Windows 11.

CPU	Arquitetura	Núcleos	Clock básico	Clock Turbo	Dram	suporte PCIe
AMD Ryzen™ 7 4800H	Zen 2	8 núcleos e 16 threads	2.9 GHz	Até 4.2 GHz	8GB	3.0
INTEL i3-10105F	Comet Lake	4 núcleos e 8 threads	3.7 GHz	Até 4.4 GHz	8GB	3.0
INTEL 5-12400F	Alder Lake	6 núcleos e 12 threads	2.5 GHz	Até 4.4 GHz	32GB	5.0 e 4.0

GPU	Arquitetura	suporte PCIe	Vram
NVIDIA GeForce GTX 1650	Turing	PCIe 3.0	4 GB
NVIDIA GeForce RTX 3080	Ampere	PCIe 4.0	10 GB
GPU NVIDIA GeForce RTX 3060	Ampere	PCIe 4.0	12 GB

Figura 9: Configurações de hardware A, B e C utilizadas no benchmark CPU (modelo, núcleos físicos, *threads* e *clock* base), memória RAM (capacidade e padrão) e GPU (modelo, CUDA cores, VRAM e geração PCIe). Diagrama feito no draw.io a partir das especificações oficiais das fabricantes (AMD, Intel e NVIDIA).

Software. Compilador GCC com OpenMP 5.0 para o módulo CPU e NVIDIA CUDA Toolkit 12.x (NVCC) para o módulo GPU. As *flags* de compilação serão `-O3 -march=native -fopenmp` no GCC e `-arch=sm_75` (Configuração A, Turing) ou `-arch=sm_86` (Configurações B e C, Ampere) no NVCC, garantindo que cada binário utilize as instruções nativas da arquitetura onde a simulação será executada.

Ferramentas adicionais. Os modelos Claude IA (Opus 4.7 e Sonnet 4.6) serão utilizados como apoio devido ao tempo limitado disponível para o desenvolvimento da simulação. O foco deste trabalho está totalmente no entendimento acadêmico análise da simulação, conceitos de arquitetura, desenvolvimento do artigo, entre outros com a finalidade de acelerar o processo mantendo a obrigação de revisão humana, realizando testes e ajustes.

6.2 Estrutura do Benchmark

O benchmark será desenvolvido utilizando a linguagem C/CUDA e terá a seguinte estrutura, podendo não se limitar a ela:

- **generator:** gerará dados fictícios: amostras gaussianas pela transformada polar de Box-Muller (a forma mais simples de descrever essa técnica é imaginar um plano cartesiano no

qual o valor de x e y serão gerados aleatórios — o ponto no plano será descartado caso saia da área definida); *timestamps* obedecendo a um processo de Poisson, deixando a simulação com cenários reais; *checksum* XOR para garantir uma verificação de paridade de forma rápida e simples.

- **search / search_cuda**: — implementação de busca linear, binária e por *hash* nas variantes serial, paralela e GPU; nesses arquivos de código será onde usaremos as APIs OpenMP e CUDA.
- **sim_math / sim_math_cuda**: implementarão Monte Carlo (estimativa de π por amostragem em $[-1, 1]^2$) e Mandelbrot (iterações até divergência).
- **benchmark.c**: atuará como orquestrador: lerá `hardware.cfg`, gerará os dados, executará todos os módulos sob cada volume e exportará os resultados em JSON.
- **hardware.cfg**: arquivo de configuração relacionado ao hardware no qual a simulação irá funcionar. Resumindo, é uma interface em formato de arquivo para configurar os parâmetros da simulação, tais como o número de núcleos, *threads* da CPU e a quantidade de CUDA cores presentes na GPU.

```
typedef struct {  
    uint32_t id; /* identificador único sequencial */  
    int64_t timestamp; /* Unix timestamp em milissegundos */  
    float valor; /* leitura do sensor - distribuição N */  
    float valor_secundario; /* segunda leitura - correlacionada */  
    Categoria categoria; /* enum 0-3 com distribuição não uniforme */  
    uint8_t status; /* 0=ok 1=aviso 2=falha */  
    char tag[32]; /* ex: "EVT-004821" - string com padrão */  
    char origem[16]; /* ex: "SENSOR_07" - origem do evento */  
    uint32_t checksum; /* XOR simples dos campos - integridade */  
    uint8_t _pad[8]; /* padding para alinhar em 96 bytes */  
} Event;
```

Figura 10: Definição da estrutura `Event` alinhada em 96 bytes.

Estrutura `Event`. Foi definido que os dados gerados vão ser baseados em informações de sensores de forma genérica. Não foi especificado o que o sensor está medindo, mas possui qualidade nas medidas utilizando Box-Muller. No caso do *timestamp* definido na *struct*, ele é gerado utilizando Processo de Poisson, considerando cenários caóticos. Uma forma simples de explicar isso é através de latência de leitura ou transferência de dados entre sensores: como exemplo, consideramos que determinado sensor possui uma leitura de dados a cada 40 ms; no mundo real, essa latência pode sofrer variações, podendo alcançar 50 ms ou 30 ms. Isso é o cenário caótico que, diferente de ambientes controlados, a realidade possui variações. Essa

simulação fará uso do Processo de Poisson para manter uma simulação mais profunda sobre sensores considerando esse cenário caótico; grande parte dos dados gerados vão se manter centralizados, porém com variações.

Alinhamento de memória (diagrama). A estrutura `Event` não possui 96 *bytes* por acaso. Na arquitetura NVIDIA, uma solicitação de carregamento para *cache* é feita em transferências de 128 *bytes*, independente se o grupo de *threads* requisita 1, 4, 9, 21... *bytes*; essa linha de transferência é dividida em 4 setores de 32 *bytes*. A estrutura `Event` foi pensada na otimização e encaixe perfeito conforme a demanda: essa estrutura consome exatamente 3 setores de 32 *bytes*, e o 4º setor já carrega a primeira parte da próxima estrutura. Caso a estrutura ocupasse 100 *bytes*, isso afetaria toda a segmentação de transferência, gastando mais linhas de *cache* de forma desorganizada. A figura a seguir demonstra isso [8] [4].

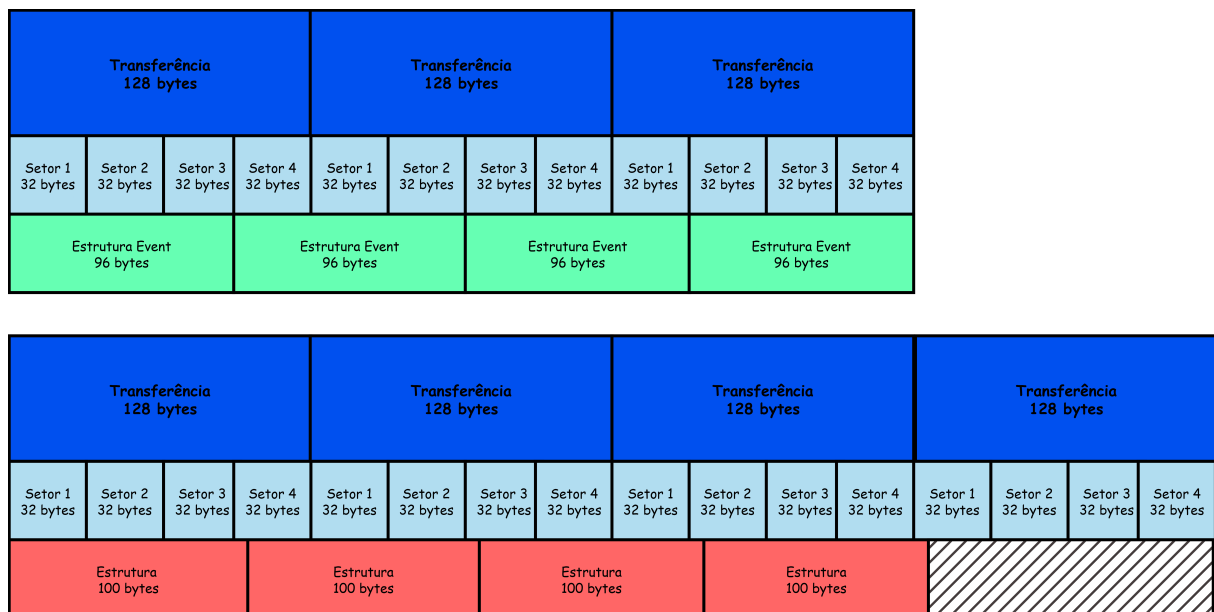


Figura 11: Ilustração do encaixe da estrutura na linha de transferência (feito no draw.io).

6.3 Procedimento Experimental

1. Geração de dados reprodutível. Para cada volume de dados gerados sinteticamente, o módulo `generator` produzirá os dados a partir de uma *seed* fixa. Isso garante que execuções diferentes processem exatamente a mesma entrada, eliminando a variação dos dados como fonte de erro e mantendo a simulação com qualidade maior.

2. Cronometragem em alta resolução. A simulação fará uso do *Time Stamp Counter* do processador, sem passar pelo relógio do sistema operacional, garantindo maior precisão. Para o módulo CUDA será utilizado o `cudaEvent_t` disponível pela própria tecnologia [4]; essa segunda medida não inclui custos como `cudaMemcpy`, apenas o processamento na GPU. A diferença entre o Tempo total e o Tempo *kernel* revela o tempo de transferência de dados no barramento PCIe tempo esse que se comporta como parte serial da lei de Amdahl e que se

mantém independente da quantidade de processadores [7][11].

3. Warmup e mediana. Cada medição seguirá um roteiro: a simulação será executada uma vez e os resultados gerados serão descartados com a finalidade de esquentar o *cache*, mantendo tudo nivelado. Em uma analogia mais simples, é como se carros de corrida fossem disputar esquentar *cache* seria o mesmo que manter o carro ligado; caso alguns carros estejam desligados e outros ligados, isso gera um gargalo no nivelamento para avaliar qual é mais veloz. Após aquecer o *cache*, a simulação fará N repetições definidas em `hardware.cfg`, das quais os resultados de saída são comparados e é escolhida a mediana. Essa decisão foi tomada pensando que processos do Windows como uma simples *swap* podem comprometer toda a simulação, descartando a possibilidade de usar a média entre as simulações. Com isso em mente, será usado o critério de refazer várias simulações e comparar os resultados de saída identificando padrões e descartando resultados que não se alinham, usando como base o seguinte raciocínio: se determinada configuração de hardware mantém um padrão em vários gráficos *speedup* $\times$ volumes de dados e outros resultados fogem 5% desses padrões, eles podem ser descartáveis. Mesmo que o artigo seja acadêmico, tudo isso tem a finalidade de manter uma excelente qualidade nas fontes.

4. Métricas derivadas. Para cada combinação (algoritmo, modo e volume) serão calculados o *speedup* observado e a eficiência por núcleo. No caso do CUDA, dois *speedups* serão calculados: o *speedup* total (incluindo a cópia de dados para a memória da GPU) e o *speedup* do *kernel* (apenas a execução na GPU). A diferença entre eles demonstra o custo do barramento PCIe.

6.4 Análise dos Dados

Os arquivos gerados pelo orquestrador, definidos até o momento como JSON, serão consumidos por um *dashboard* HTML interativo gerado por IA. Pode também ser considerado o processamento de gráficos gerados por Python, contendo gráficos como *speedup* por volume e *ranking* por algoritmo. Conforme acessamos esses gráficos, podemos expandir a análise e fazer observações importantes, gerando novos gráficos não citados anteriormente.

A análise também será baseada na comparação entre os dados gerados pelo benchmark e previsões teóricas feitas pelas leis de Amdahl e Gustafson.

A lei de Gustafson será aplicada de forma complementar: ao crescer o volume de dados, espera-se que o *speedup* observado aumente (em vez de saturar como Amdahl prevê para problema fixo). Algoritmos cujo *speedup* continua crescendo com o volume são classificados como **compute bound** são os candidatos onde a GPU realmente brilha. Algoritmos cujo *speedup* satura cedo são classificados como **memory bound** onde a transferência domina e a GPU pode até perder para a CPU paralela [6].

Essa classificação dupla (qual curva de Amdahl encaixa + cresce ou satura com volume) é o que conecta os números medidos às hipóteses do trabalho: H1 prevê inversão de regime na

busca linear conforme o volume cresce, H2 prevê degradação na busca por *hash* com colisões, e H3 prevê que matemática densa fica próxima do limite de Gustafson. A análise permitirá responder se cada hipótese se confirmou, parcialmente confirmou ou não se confirmou.

7 Resultados Esperados

- Em Monte Carlo e Mandelbrot, espera-se que a GPU supere a CPU serial entre $50\times$ e $500\times$, em conformidade com a lei de Gustafson (regime *compute bound*) [7].
- Na busca linear sobre grandes volumes, espera-se vantagem da GPU entre $5\times$ e $50\times$.
- Em busca binária e *hash* com volumes pequenos, espera-se que a CPU paralela vença a GPU devido à latência PCIe e ao regime *memory bound*, evidenciando o teto previsto pela lei de Amdahl.
- Espera-se que o estudo forneça subsídios práticos para a escolha de hardware/algoritmo conforme o tipo de carga.

8 Cronograma

O cronograma cobre o ciclo completo do projeto, organizado em **seis semanas** entre 13/04/2026 e 22/05/2026 (entrega final).

Sobre o protótipo do benchmark. A implementação em C/CUDA presente em `projeto_benchmark/simulacao/` é **apenas um protótipo**, desenvolvido como **andaime metodológico** para validar a viabilidade da estrutura `Event`, da geração sintética e da orquestração $CPU \leftrightarrow GPU$, dando entrada à análise do valor que este artigo de **proposta** se compromete a entregar. A execução completa do benchmark, com varredura de volumes, repetições, mediana e resultados medidos, fica reservada à Etapa 3 (relatório final).

Sobre as ferramentas de apoio. Conforme declarado na Seção 6.1, ferramentas auxiliares incluindo modelos de IA foram utilizadas para tarefas operacionais (rascunho de código do protótipo, geração de materiais internos de estudo, organização de listas e tabelas), sempre sob direção e revisão dos autores e sem produzir conteúdo intelectual do artigo.

Tabela 1: Cronograma de execução do projeto.

Semana	Período	Atividades
1	13/04 – 19/04	Abertura do projeto e definição das ferramentas. Levantamento e convergência do problema de pesquisa (leis de Amdahl e Gustafson em diferentes cenários). Fechamento da arquitetura geral (módulos serial, OpenMP, CUDA; estrutura <code>Event</code> de 96 B; orquestrador). Início da codificação do protótipo (<code>event.h</code> , <code>generator</code>).
2	20/04 – 26/04	Codificação completa do protótipo: módulos <code>search</code> (CPU), <code>sim_math</code> e <code>sim_math_cuda</code> (Monte Carlo e Mandelbrot), <code>search_cuda</code> , orquestrador <code>benchmark.c</code> , <code>Makefile</code> , <code>build_cuda.bat</code> , <code>dashboard.html</code> , <code>guide.html</code> . Compilação dos distribuíveis para Turing (sm_75) e Ampere (sm_86). Primeiras execuções nas Configurações A e B. Conclusão das frentes de nivelamento (Paralelismo, CUDA Core, <i>threads</i> em C, otimização serial). Coautor anexou referências ao Google Acadêmico.
3	27/04 – 03/05	Inclusão do modelo Sun–Ni na fundamentação teórica. Organização de material interno de estudo (documentação modular do projeto, animações didáticas, fichas de leitura) — uso exclusivamente acadêmico do autor, não compõe o texto deste artigo. Decisão pela redação do artigo no Overleaf e migração de tarefas residuais (repositório GitHub, ajustes finos do motor) para a Etapa 3. Consolidação das fontes normativas (CUDA C++ Programming Guide; OpenMP API 5.2). Coautor anexou novas referências.
4	04/05 – 10/05	Início e desenvolvimento da redação no Overleaf: Seções 1 (Introdução), 2 (Problema de Pesquisa), 3 (Hipóteses) e 4 (Objetivos).

Semana	Período	Atividades
5	11/05 – 17/05	Continuidade da redação no Overleaf: Seção 5.3 (CUDA e OpenMP), Seção 6.1 (Materiais), Seção 6.2 (Estrutura <code>Event</code> e Alinhamento de memória), Seção 6.3 (procedimento experimental — <i>bullets</i> 1, 3 e 4) e Seção 6.4 (primeiro parágrafo).
6	18/05 – 22/05	Anexação do segundo lote de referências para a Seção 5.4 (Trabalhos Relacionados). Redação das pendências finais: Seções 5.1 (Lei de Amdahl), 5.2 (Lei de Gustafson) e 5.4 (Trabalhos Relacionados); fechamento dos trechos pendentes em 6.2 (<i>coalescing</i> 96 B vs 100 B), 6.3 (<code>cudaEvent_t</code> e PCIe) e 6.4 (análise dos dados). Fechamento da Seção 9 (referências em ABNT NBR 6023). Revisão integral, geração do PDF e entrega em 22/05/2026 .

9 Referências

Referências

- [1] AL-HAYANNI, M. A. N. *et al.* **Amdahl’s law in the context of heterogeneous many-core systems – a survey**. *IET Computers & Digital Techniques*, v. 14, n. 4, p. 133–148, 2020. Disponível em: <https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/iet-cdt.2018.5220>.
- [2] **Modernizing Amdahl’s Law: How AI Scaling Laws Shape Computer Architecture**. *arXiv preprint* arXiv:2603.20654, 2026. Disponível em: <https://arxiv.org/abs/2603.20654>.
- [3] WANG, Y. *et al.* **Parallel Paradigms in Modern HPC: A Comparative Analysis of MPI, OpenMP, and CUDA**. *arXiv preprint* arXiv:2506.15454, 2025. Disponível em: <https://arxiv.org/abs/2506.15454>.
- [4] NVIDIA CORPORATION. **CUDA C++ Programming Guide**. Versão 12.x. Santa Clara: NVIDIA, 2024. Disponível em: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [5] KHALILOV, M.; TIMOVEEV, A. **Performance analysis of CUDA, OpenACC and OpenMP programming models on Tesla V100 GPU**. *Journal of Physics: Conference*

- Series*, v. 1740, art. 012056, 2021. Disponível em: <https://iopscience.iop.org/article/10.1088/1742-6596/1740/1/012056>.
- [6] BENMERZOUG, M. *et al.* **A Comparative Study of OpenMP Scheduling Algorithm Selection Strategies.** *arXiv preprint* arXiv:2507.20312, 2025. Disponível em: <https://arxiv.org/pdf/2507.20312>.
 - [7] LIN, B. *et al.* **Efficient GPU implementation of the Particle-in-Cell/Monte-Carlo collisions method for 1D simulation of low-pressure capacitively coupled plasmas.** *Computer Physics Communications*, Elsevier, v. 261, art. 107778, 2021. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0010465521000503>.
 - [8] LI, X. *et al.* **cuThermo: Understanding GPU Memory Inefficiencies with Heat Map Profiling.** *arXiv preprint* arXiv:2507.18729, 2025. Disponível em: <https://arxiv.org/html/2507.18729v1>.
 - [9] CASTRO, M. *et al.* **Heterogeneous Computing for High-Complexity Problems: A Performance Study with DPC++ on HPC Platforms.** *Data and Metadata*, v. 2, 2023. Disponível em: <https://dm.ageditor.ar/index.php/dm/article/view/835>.
 - [10] OPENMP ARCHITECTURE REVIEW BOARD. **OpenMP Application Programming Interface — Version 5.2.** OpenMP ARB, 2021. Disponível em: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>.
 - [11] VÉGH, J. **Re-evaluating scaling methods for distributed parallel systems.** *arXiv preprint* arXiv:2002.08316v2, 2020. Disponível em: <https://arxiv.org/abs/2002.08316>.
 - [12] SUN, X.-H. *et al.* **The Memory-Bounded Speedup Model and Its Impacts in Computing.** *Journal of Computer Science and Technology*, v. 38, n. 1, p. 64–79, 2023. Disponível em: <https://jcst.ict.ac.cn/fileup/1000-9000/PDF/JCST-2023-1-4-2911-64.pdf>.
 - [13] AMDAHL, G. M. **Validity of the single processor approach to achieving large scale computing capabilities.** In: *AFIPS Spring Joint Computer Conference*, p. 483–485, 1967. Disponível em: <https://dl.acm.org/doi/10.1145/1465482.1465560>.
 - [14] GUSTAFSON, J. L. **Reevaluating Amdahl's Law.** *Communications of the ACM*, v. 31, n. 5, p. 532–533, 1988. Disponível em: <https://dl.acm.org/doi/10.1145/42411.42415>.